

1. Project description

The project is a tool for converting documents between typesetting formats, which in its minimal version will cover GFM (GitHub Flavored Markdown) to typst and latex. The tool will provide a command-line interface for users to interact with it.

2. Subproblems and their solving methods

- Parsing engine (C++)
Problem: Transformation of one typesetting format into a structured object model
Solution: Use the libcmarm-gfm library (the official GitHub Flavored Markdown parser) to transform raw text into an initial C-based AST. Then, map this structure onto a custom, high-level C++ Class Hierarchy (Object Model) for easier manipulation.
- Rendering system (C++)
Problem: Converting the AST into the specific syntax of LaTeX and Typst in an extensible manner.
Solution: Implement all Writers directly in C++ using the Visitor pattern. This ensures that the AST remains internal and protected from external modification, providing a high-performance "black-box" conversion engine.
- Language Integration
Problem: Providing a user-friendly CLI while keeping the heavy logic in C++.
Solution: Use pybind11 to create a thin Python wrapper. Python will handle argument parsing and call a single "Dispatcher" function in C++, which triggers the internal parsing and rendering pipeline.
- User Interface (Python)
Problem: Providing an easy experience for users
Solution: The program will accept source file paths, target format flags, and optional output destinations.
- Error handling
Problem: Handling invalid inputs, unsupported operations, and C++ to Python error propagation
Solution: When the C++ core throws exceptions, the Python wrapper will catch them and translate them to user-readable CLI error messages.

3. Project's functionalities

Given the project's time constraints, the Minimum Viable Product will focus on the reliable conversion of the following elements:

- Headings (H1 - H3)
- Paragraphs + line breaks
- Bold text (**text**)
- Italic text (*text*)
- Inline code (`code`)
- Fenced code blocks (

```
lang
```

)
- Unordered lists (- item)
- Ordered lists (1. item)
- Links ([text](url))

- Thematic breaks (---)
- Blockquotes (> text)
- Images (![alt](src))
- Tables
- Nested lists

4. Architecture

The core logic of the tool will be implemented in C++23, and the CLI will be completed in a Python module, which should be unaware of the actual technicalities inside the code and only provide an entry point for the users to invoke.

The implementation will be inspired by pandoc, having a parser -> syntax tree -> writer architecture.

The project requirements suggest potential compatibility with the Pandoc AST format. After evaluation, full Pandoc AST compatibility was excluded from the MVP scope. Instead, a custom C++ Object Model is used, which is purpose-built for the supported formats and is easier to extend incrementally.

5. Technological stack

- C++ 17 - conan, cmake
- Python 3.11 - Python uv
- pybind11 for Python - C++ integration
- Tests - C++ Catch2, python pytest
- Formatters - C++ clang, python black